

4. Sockets

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

4.1 Vorschau

- Grundlagen
- Sockets
- Skalierbare Server-Architektur
 - Load-Balancer
 - Multi-Threading
 - Asynchrone Sockets
- Bibliotheken
- Zusammenfassung



4.1 Grundlagen



4.1 Grundlagen

- Cloud-Umgebung:
 - hohe Zuverlässigkeit (kaum Paketverluste, selten Partitionierungen)
 - vergleichsweise sichere Umgebung
- P2P/Internet:
 - geringere Zuverlässigkeit
 - wenig vertrauenswürdig
- Netzwerkcharakteristika

Netzwerk	Bandbreite (max.)	Latenz (min.)
LAN	400 GBit/s	900ns
Internet	1 GBit/s	1-100ms
WLAN	6,9 GBit/s	1ms
LTE (advanced)	1 GBit/s (downlink)	5ms



Protokolle aus Sicht von Java

Java APIs

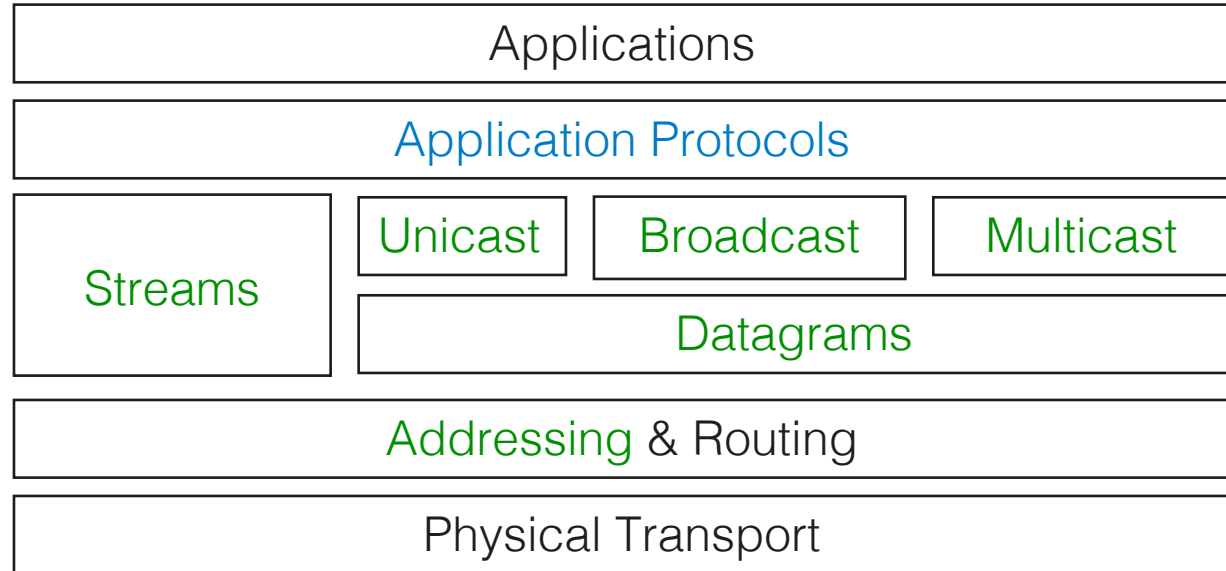
http, custom

1:1 or 1:Many

TCP, UDP

Internet Protocol

Ethernet



Transfer Control Protocol (TCP)

- **Streams:** Nachrichten werden als Bytestrom versendet
 - Adressierung erfolgt durch eine Verbindung zwischen zwei Endpunkten
 - Erhält Sequenz der Nachrichten
 - Verlorene Nachrichten werden automatisch neu übertragen
- Weitere Aspekte (siehe Rechnernetze):
 - Flusskontrolle (engl. flow control): vermeidet Überlastung des Empfängers
 - Verstopfungskontrolle (engl. congestion avoidance): vermeidet Verstopfung im Netzwerk → Router kann überlastet sein, falls viele Verbindung über ihn gehen
- Beispiele: https, ssh, sftp, X11, ...



User Datagram Protocol (UDP)

- Datagrams: Nachrichten werden als Datagramme verschickt
- Verbindungslos
- Nachrichtensequenz nicht garantiert
- Unter Umständen Nachrichtenverluste
- Keine Fluss- und Verstopfungskontrolle
- Beispiele: Video- und Audioströme, tftp, ...



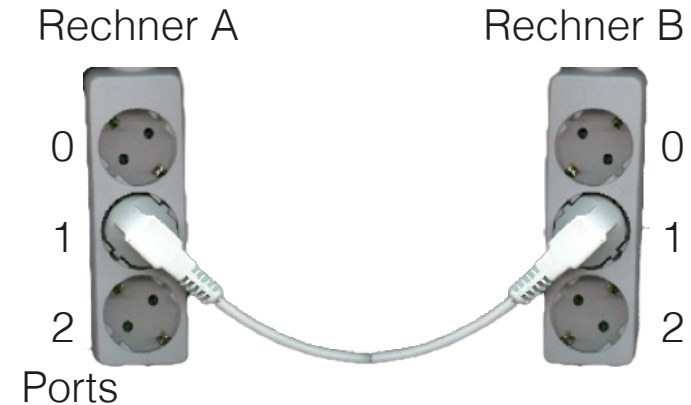
Internet Protocol (IP)

- **Aufgaben: Adressierung und Routing**
 - abstrahiert vom unterliegendem physikalischen Netz
 - einheitliche Weglenkung (engl. routing)
- **Adressen, um einen Rechner eindeutig zu identifizieren:**
 - IPv4 verwendet 32 Bit Adressen, z.B. 192.168.70.30
 - IPv6 löst Knappheit der Adressen durch 128 Bit Adressierung
 - Meist werden Domain Names verwendet
 - Domain Name System (DNS) übernimmt Abbildung auf IP-Adressen



Internet Protocol (IP)

- Port-Nummern zur Identifikation eines bestimmten Diensts auf einem Rechner
 - 16 Bit Nummer
 - Prozess/Dienst muss sich beim System auf einem Port registrieren
 - Es gibt reservierte und frei verfügbare Ports (Programm kann es dem Betriebssystem überlassen einen freien Port zuzuordnen)
- Beispiele reservierter Ports:
 - Ports 22 SSH
 - Port 25 SMTP
 - Port 80 HTTP
 - Port 143 IMAP



4.2 Sockets



4.2 Sockets

- Sockets sind Kommunikationsendpunkte
- Für eine nachrichtenbasierte Kommunikation zwischen Prozessen/Rechnern
- Für Java nur Internet-Protokollarchitektur
- (in C für „irgendeinen“ Protokollstack)



IP-Adressierung

- Java hat hierfür die Klasse `java.net.InetAddress`
- Hat drei Aufgaben:
 - Kapselt eine IP-Adresse
 - Kann einen DNS-Lookup durchführen (DomainName → IP-Adresse)
 - Bietet auch einen ReverseDNS-Lookup (IP-Adresse → DomainName)
- Auszug Methoden zum Erzeugen von `InetAddress`-Instanzen:
 - mit gegebener IP-Adresse: `InetAddress getByAddress(byte[] addr)`
 - mit DNS-Namen: `InetAddress getByName(String hostName)`
 - Localhost: `InetAddress getLocalHost()`



UDP - Beispiel

- Client und Server verwenden zum Senden/Empfangen die Klasse `DatagramSocket`
- Pakete sind Instanzen der Klasse `DatagramPacket`
- IP-Adresse und Port-Nummer in jedem Paket (da keine Verbindung vorhanden)
- Daten in Byte-Array



Beispiel: Server (Java UDP)

```
import java.io.*;    // IO exceptions
import java.net.*;   // sockets

public class Server {

    public static void main(String[] args) {
        DatagramSocket socket; // UDP socket
        DatagramPacket packet; // UDP packet
        String str;
        byte data[] = new byte[256];

        try {
            socket = new DatagramSocket(4711);
            while (true) {
                packet = new DatagramPacket(data, data.length);

                // blocking receive
                socket.receive(packet);
                str = new String(packet.getData(), 0, packet.getLength());

                System.out.println("received: "+str);
            }
            socket.close();
        } catch (IOException e) {}
    }
}
```



Beispiel: Client (Java UDP)

```
import java.net.* ; // sockets
import java.io.*;   // exceptions

public class Client {

    static String url = "localhost";
    static int port = 4711;

    public static void main (String[] args){
        byte[] data = "VerteilteSysteme".getBytes();

        try {
            DatagramSocket socket = new DatagramSocket();
            DatagramPacket packet = new DatagramPacket( data, data.length, InetAddress.getByName(url), port);

            socket.send( packet);

            socket.close();
        }
        catch (IOException e) {
        }
    }
}
```



TCP in Java

- Client:
 - Verwendet die Klasse `Socket`
 - Konstruktor erhält IP-Adresse und Port-Nummer
 - Verbindungsaufbau beim Anlegen der `Socket`-Instanz
- Server:
 - Verbindungsannahme mit Klasse `ServerSocket` (an Port gebunden)
 - `accept()` blockiert, wartet auf Verbindung und liefert neuen `Socket` für die eigentliche Kommunikation



Beispiel: Simple EchoServer (Java TCP)

```
import java.net.*; import java.io.*;

public class EchoServer {
    public static void main(String[] s){
        try {
            ServerSocket sock = new ServerSocket( 4711 );
            Socket conn = sock.accept(); // block until connection request
            OutputStream out = conn.getOutputStream();
            InputStreamReader inp = conn.getInputStream();

            out.println( "Echo server active, enter 'bye' to exit.\r");
            do {
                idx = 0;
                do { // read one line, byte by byte
                    buffer[idx] = (byte) inp.read(); // read one byte
                    out.write( buffer[idx] );
                    idx++;
                } while(buffer[idx-1] != '\r');

                String msg = new String(buffer, 0, idx-1); // convert raw bytes to String
                System.out.println("received: " + msg);
                if ( msg.indexOf("bye") > 0 )
                    break;
            } while( true );
            conn.close( );
        } catch ( IOException ioX ) { ...
```



Byte Order

- Die Reihenfolge in welcher die Bytes von Datentypen gespeichert werden, welche aus mehr als ein Byte bestehen, ist abhängig vom Prozessor.
 - Auch als **Host Byte Order** bezeichnet
 - x86-basierte Prozessoren verwenden Little Endian Byte Ordering
- TCP/IP verwendet hingegen Big Endian Byte Ordering
 - Auch als **Network Byte Order** bezeichnet
- Beispiel: 0xCAFE (32-Bit, uint₃₂)

Big Endian Byte Ordering

Offset	0	1	2	3
Content	E	F	A	C

Little Endian Byte Ordering

Offset	0	1	2	3
Content	C	A	F	E



Byte Order (2)

- Java-Streams verwenden Big Endian Byte Ordering
→ entspricht Network Byte Order
- Falls der Client und Server in Java geschrieben ist, müssen wir bzgl. der Byte-Order nichts beachten
- Falls wir Sockets in C programmieren muss die Byte Order sowieso immer beachtet werden!
 - Die C-Socket-Bibliothek stellt Funktionen zur Konvertierung zwischen Host- und Network Byte Order bereit:

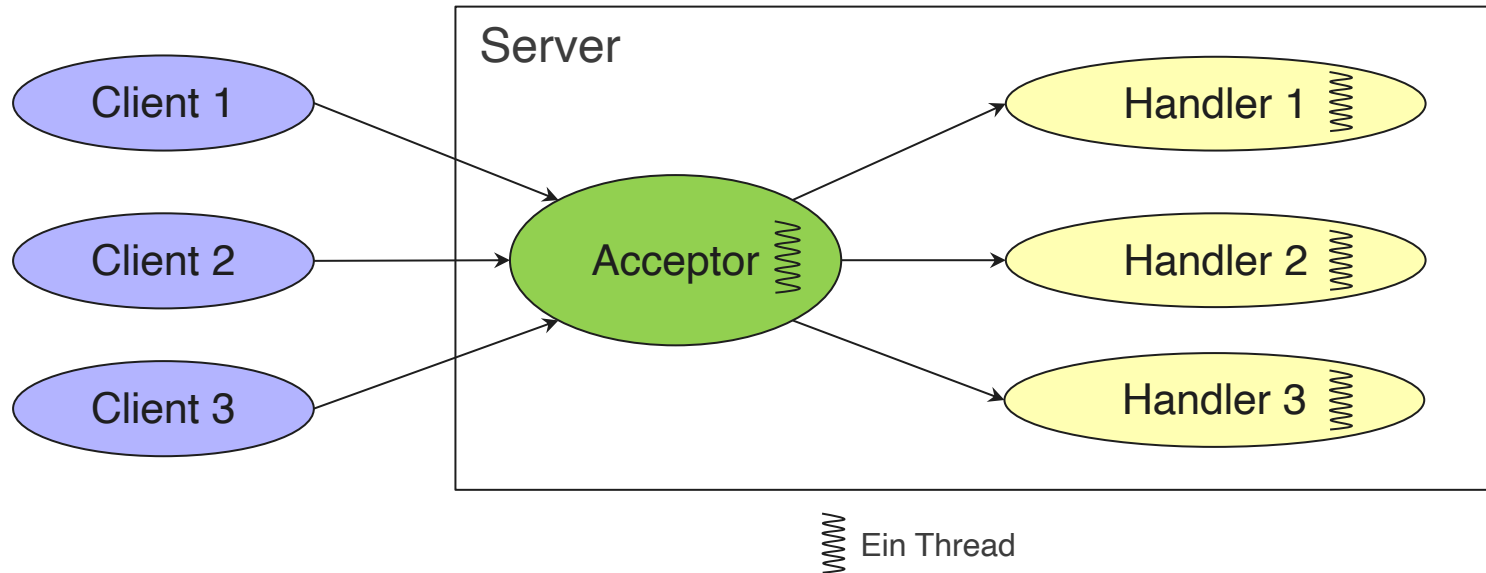
```
long  htonl (long value);      /* host to network long */
short htons (short value);     /* host to network short */
long  ntohl (long value);      /* network to host long */
short ntos  (short value);     /* network to host short */
```

- Achtung: auch bei der IP-Adresse und Port-Nummer beachten



Multi-Threaded Server für mehrere Clients

- Architektur: ein Thread pro Handler + ein Thread für den Acceptor



Multi-Threaded EchoServer (abstrakt)

- Acceptor-Thread, Klasse EchoServer:

```
ServerSocket sock = new ServerSocket( 4711 );

do {
    Socket conn = sock.accept();           // block until connection request
    Handler handler = new Handler( conn );
    handler.start();                       // start handler thread
} while( true );
```

- Handler-Thread, Klasse Handler:

```
public void run() {
    OutputStream out = conn.getOutputStream();
    InputStreamReader inp = conn.getInputStream();

    out.println( "Echo server active, enter 'bye' to exit.\r" );
    do {
        do { // ...
        } while( buffer[idx-1] != '\r' ); // read one line, byte by byte
        // ...
    } while( true );
}
```



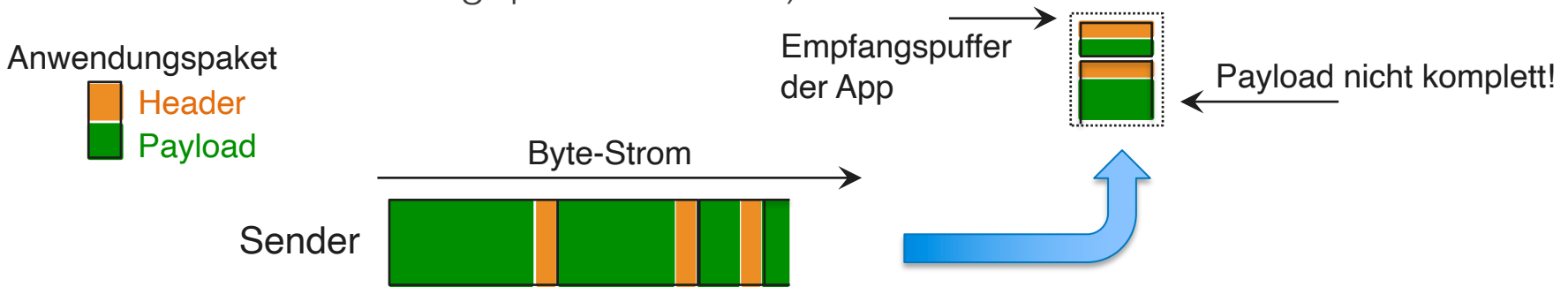
Aufbau von Anwendungsnachrichten

- Für jede Anwendung muss ein **Protokoll** entworfen werden
→ Format der auszutauschenden Daten
- Basisformat:
Header (feste Länge): Paket-Typ (bestimmt Format der Daten) + Länge des Payload
Payload (Daten mit variabler Länge)
- Sender wandelt Java-Objekte in Nachrichten um und verschickt diese.
- Empfänger muss Nachrichten aus dem Bytestrom erkennen und wandelt diese dann wieder in Java-Objekte um
- Für das Umwandeln von Objekten in Byte-Streams und zurück gibt es in Java das Interface `Serializable` (ist aber langsam)



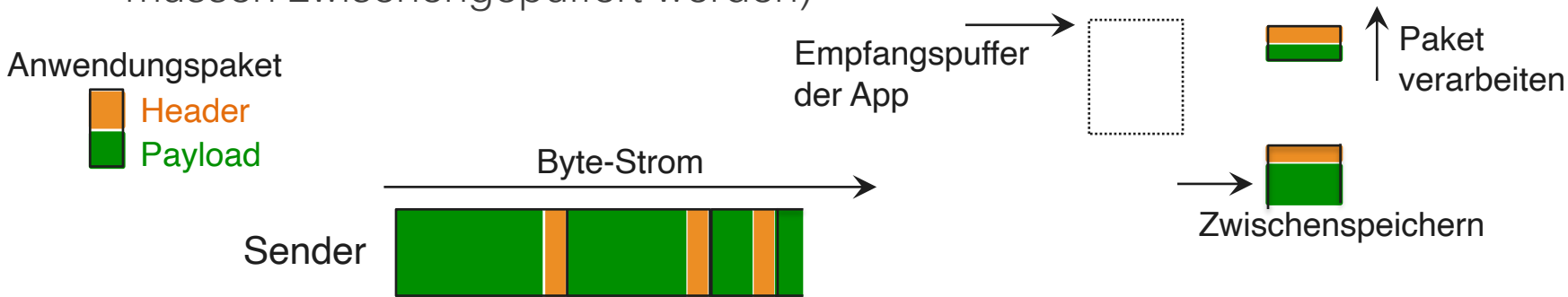
TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



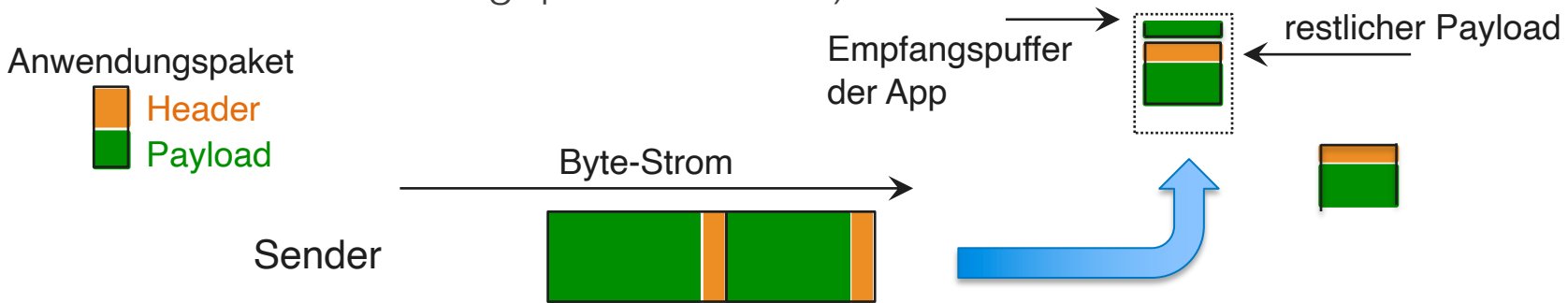
TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



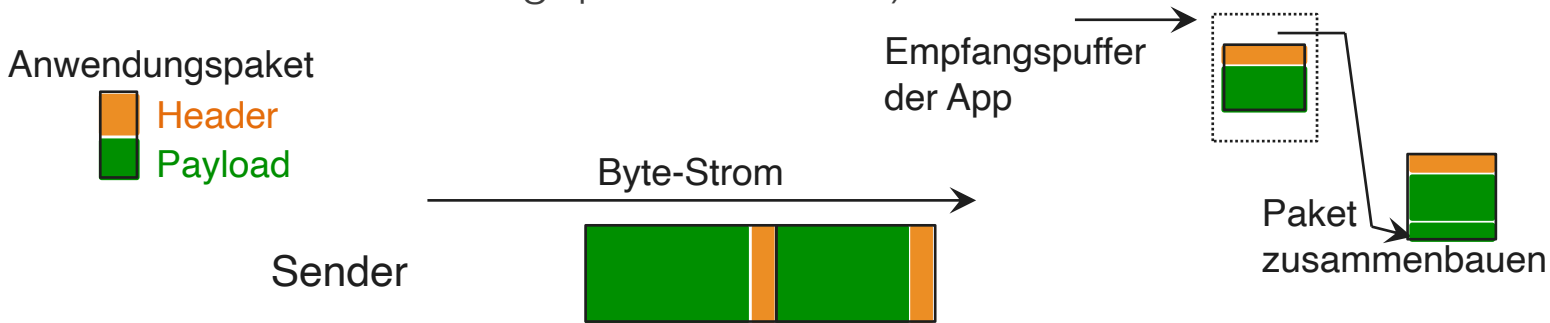
TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



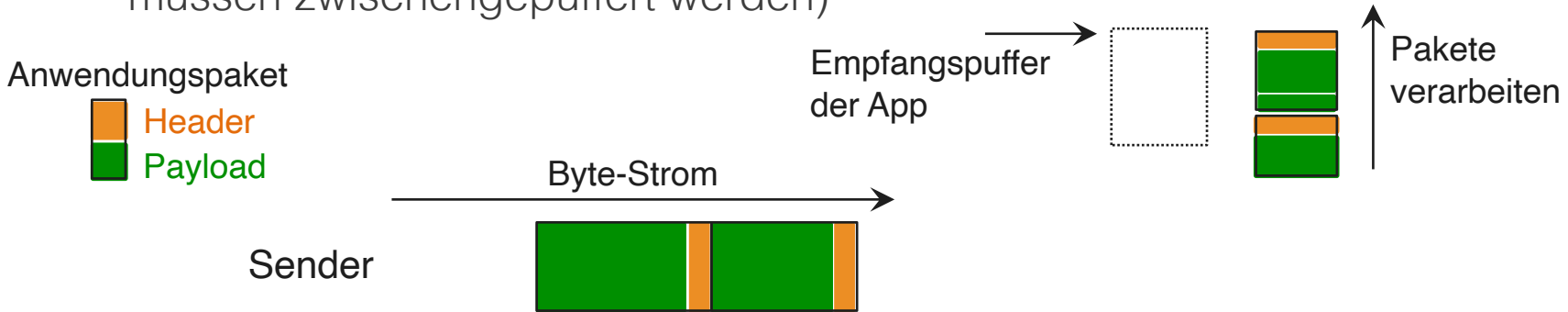
TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



Anfrage-Antwort Zuordnung

- Bei **Request-Responses** (Anfrage/Antwort) wird typischerweise eine ID verwendet, um eine eindeutige Verknüpfung zu gewährleisten
 - Eventuell werden zwischen und Request noch andere Nachrichten empfangen, sodass der Empfänger entsprechend verwalten muss, welche Anfragen ausstehen und dann bei Eingang einer Antwort diese richtig zuordnen und dann weiter verarbeiten kann
- Dieses Verhalten gibt es zum Beispiel beim X-Protokoll (X Server)
 - Der X Server schickt ereignisgetrieben zum Beispiel Maus-Ereignisse
 - Und die Anwendung schickt Zeichenbefehle zum X Server und ggf. auch Abfragen mit Antworten
 - Dies geschieht alles asynchron



4.3 Skalierbare Server-Architektur



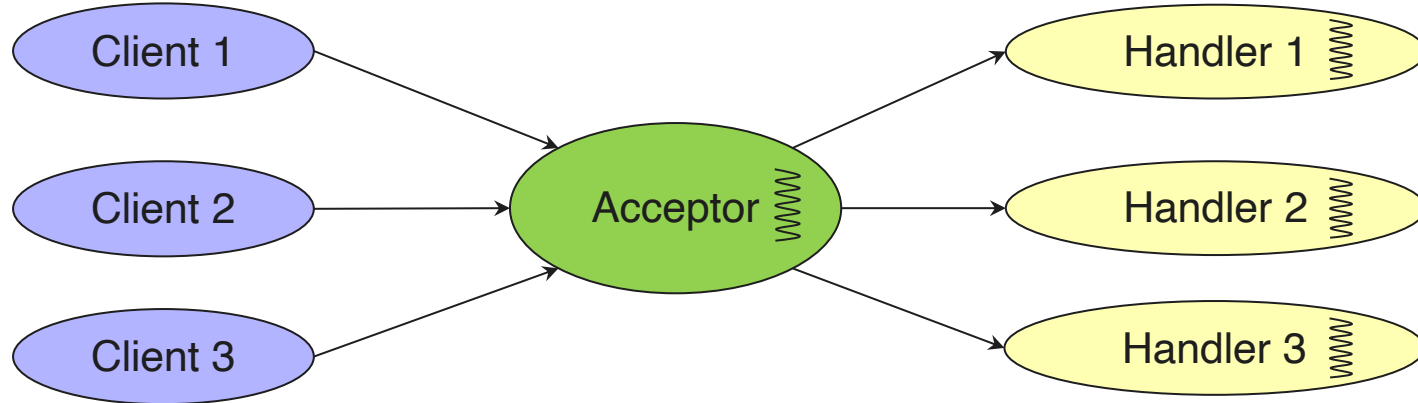
Server für mehrere Clients

- Typischerweise muss ein Server gleichzeitig mehrere Clients bedienen (für jeden Client eine gibt es eine blockierende Verbindung)
- Einfache Lösung
 - Haupt-Thread wartet in einer Schleife blockierend auf eingehende Verbindungswünsche → **Acceptor**
 - Für jede neue Verbindung erzeugt er einen neuen Thread der den neuen Client/Socket blockierend bedient → **Handler**



Multi-Threaded Server für mehrere Clients

- Architektur: ein Thread pro Handler + ein Thread für den Acceptor



- Bei vielen Verbindungen würde solch ein Server sehr langsam arbeiten
- Diese Architektur skaliert schlecht hinsichtlich der Anzahl an Clients**

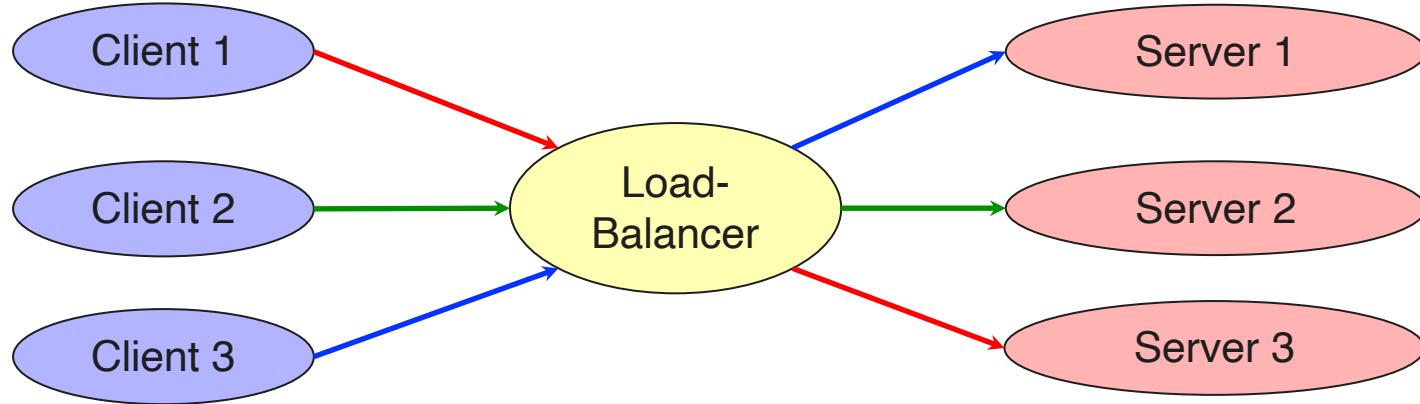
Skalierbarkeit bzgl. Anzahl Clients

- Probleme:
 - Ein Thread in einer 64-Bit JVM belegt ca. 1 MB (auch wenn er inaktiv ist)
 - Erzeugen und Beenden von Threads kostet auch Zeit
 - Viele Threads belasten auch den Scheduler
 - Aufwecken blockierter Threads dauert >10 Millisekunden
 - Umkopieren von Daten kostet zusätzlich Speicher und Zeit
- Diese Probleme treten auf, wenn ein Server viele Klienten bedienen muss und wenn ein hoher Durchsatz und kurze Latenz gewünscht ist.



Lösung 1: Load-Balancer

- Wir verwenden mehrere Server, aber alle Clients greifen über einen Load-Balancer indirekt auf einen Server zu
- Damit kann die Last auf die verschiedenen Server einfach verteilt werden



- Sehr oft verwendet in der Praxis (bietet zu dem Fehlertoleranz, durch mehrere Server, siehe später mehr)

Lösung 2: asynchrone Sockets

- **Asynchrone Sockets verwenden** → arbeiten **ereignisbasiert**
 - Pro Client nach wie vor ein Socket
 - Für den Datenempfang wird aber kein blockierendes `read` aufgerufen
 - Stattdessen registriert man sich für Ereignisse auf dem Socket und gibt eine Callback-Methode an. Diese wird aufgerufen, wenn das Ereignis eintritt.

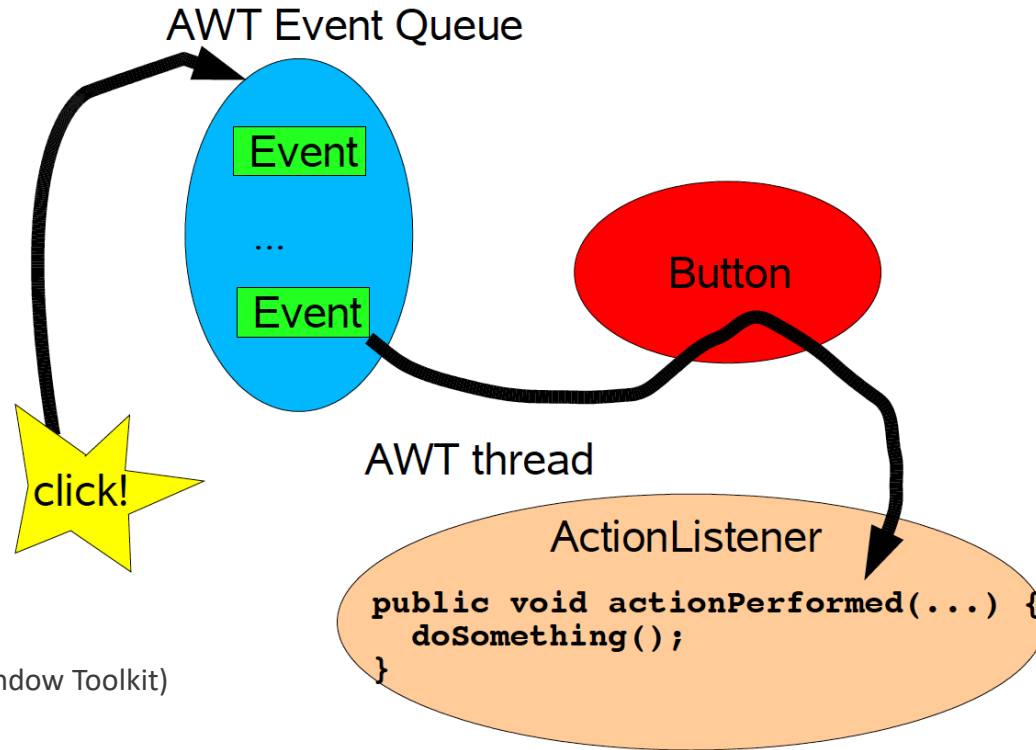


Lösung 2: asynchrone Sockets

- **Ergebnis: 1 Thread** verarbeitet die Daten von mehreren asynchronen Sockets
 - Dieser Thread holt alle Ereignisse ab, für u.U. mehrere Sockets, und ruft dann die für jedes Ereignis registrierte Callback-Methode auf
 - Damit nicht mehr ein „teurer“ Thread pro Verbindung/Client
- Aber die Verarbeitung der Nachrichten auf dem Server darf nicht blockieren, ansonsten könnte die Abarbeitung einer Nachricht auf einer Verbindung eine andere Verbindung negativ beeinflussen



Verwandte Idee bei der GUI-Programmierung

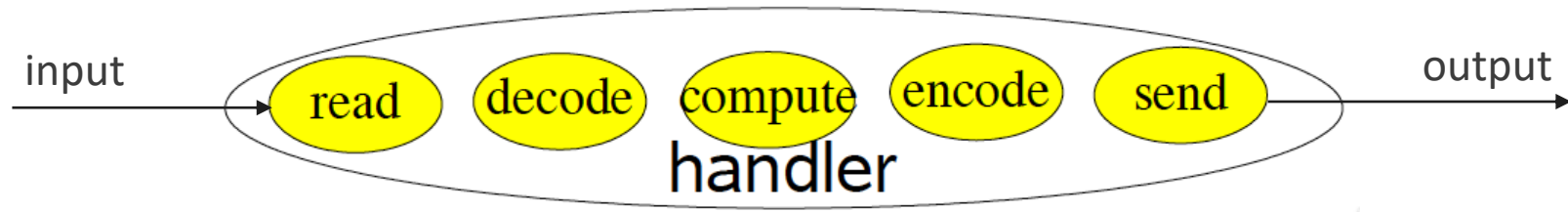


(AWT = Abstract Window Toolkit)

Doug Lea, State University of New York at Oswego

Ereignisgetriebene Server-Architektur

- Idee: unterteile Server-Verarbeitung (engl. handler) in nicht-blockierende Aufgaben (engl. tasks)
- Eine Task wird ausgeführt, wenn sie über ein Ereignis angestoßen wird
 - Kette der Tasks in einem Handler wird initial durch ein Empfangereignis angestoßen
 - Ablauf der Task-Kette (ähnlich Zustandsautomat) muss eingehalten werden
 - Programmierer muss Zustand des Handlers verwalten, um asynchrone Abarbeitung zu steuern



Doug Lea, State University of New York at Oswego

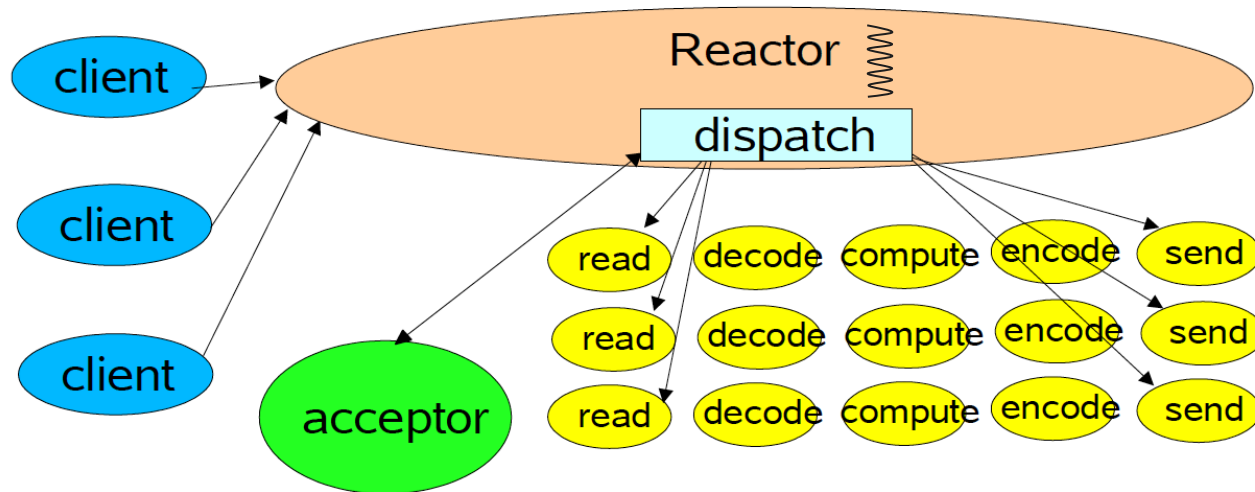
Reactor-Pattern

- Reactor reagiert auf I/O-Ereignisse indem er den zuständigen Handler ruft
 - Entspricht dem Java-AWT Thread bei der GUI-Programmierung
- Handler führen nicht-blockierende Aktionen durch
 - Vergleiche AWT ActionListeners
- Zuordnung von Handlern zu Ereignissen erfolgt Registrierungen
 - Siehe auch AWT `addActionListener`
- (AWT = Abstract Window Toolkit)



Stand bisher

- Server mit einem Thread, welche u.U. viele asynchrone Sockets bedient
- Ereignisgetriebene, zustandsabhängige Verarbeitung
- Handler oder Acceptor werden gerufen, wenn ein Ereignis eintritt, für dass sie sich registriert haben



Bewertung

- Bisheriger Ansatz mit einem Thread nutzt nicht Multicore-Hardware
→ von zu vielen Threads, sind wir zu nur einem Thread gewechselt
- Wir möchten gerne **mehrere Worker-Threads** die mehrere Handler parallel ausführen können
 - Aber nur beschränkt viele Threads!
 - Typischerweise insgesamt so viele Threads wie Cores vorhanden sind



Worker-Threads

- Ziel: Worker-Threads sollen den Reactor-Thread entlasten
 - Wir möchten aber nicht für jede Nachricht einen Threads erzeugen.
 - Das ist viel zu langsam!
- Lösung: Thread-Pool (verwaltet mehrere Threads)
 - Threads werden nach der Nutzung nicht zerstört, sondern einfach neu zugeordnet, wenn sie benötigt werden
→ schneller als jedes Mal einen Thread zu erzeugen
 - Falls die Threads nicht alle eingehenden Anfragen schnell genug abarbeiten können, so werden diese in einer Task-Queue (angehängt am Thread-Pool verwaltet und später abgearbeitet)
 - Es gibt viele Thread-Pool-Lösungen (u.a. `ThreadPoolExecutor` in Java)



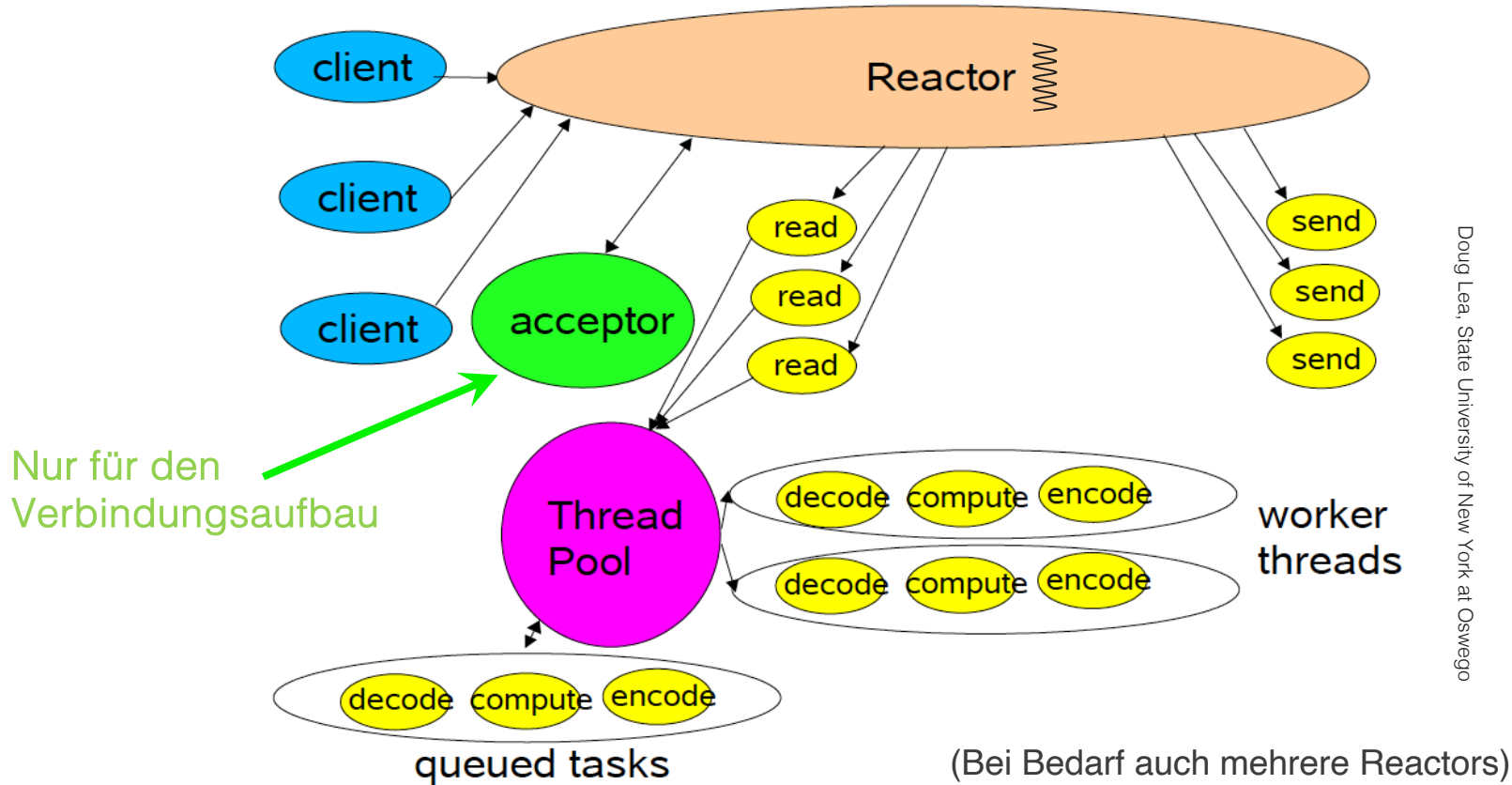
Worker-Threads

- Funktionsprinzip eines Thread-Pools ähnelt einem Restaurant.
- Anzahl der Bedienungen, die Essen zu den Gästen bringen, ist fest.
- Produziert die Küche mehr Essen als die Bedienungen ausliefern können, stauen sich die Teller temporär. Es werden aber kurzfristig keine neuen Bedienungen eingestellt.
- Ebenfalls werden keine entlassen, falls weniger Essen bestellt werden.

(Quelle: https://www.w3l.de/de/fileadmin/user_upload/Datenstrukturen_und_Grundlagen_der_nebenlaeufigen_Programmierung_in_Java_2014.pdf)



Multi-Threaded Architektur



Doug Lea, State University of New York at Oswego

Bemerkungen zur ereignisgetriebenen Server-Architektur

- Asynchrone Programmierung erfordert Umdenken gegenüber blockierenden Threads → anspruchsvoll
- Ereignisgetriebene Logik (ähnlich einem Zustandsautomat) muss sichergestellt werden
- Eine Blockierung kann nicht immer komplett umgangen werden (Garbage Collector, Locks etc.)
- Wichtige Publikation:
"SEDA: an architecture for well-conditioned, scalable internet services",
Welsh et.al., ACM SOSP 2001.



4.4 Bibliotheken



Netty

- Eine über viele Jahre optimierte Bibliothek (Open-Source) für socket-basierte Kommunikation für Java-Anwendungen
- Verwendet intern asynchrone Java NIO-Sockets & Thread-Pools
- Basiert auf einer ereignis-getriebenen Architektur
 - Hohen Durchsatz (durch automatische Aggregation von Netzwerknachrichten)
 - Sowie niedrige Latenz
- Siehe: <https://netty.io/index.html>





- Open-Source C-Bibliothek¹ für sehr schnelle Netzwerke in Rechenzentren
 - Unter anderem InfiniBand (400 Gbit/s = 50 Gbyte/s & < 0.5 μ s Latenz)
- Basiert auf einer ereignis-getriebenen Architektur
- Abstrahiert die Low-Level InfiniBand Verbs API mittels der UCP API²
 - Auch asynchrone Programmierung, aber keine Sockets
- Identifiziert und Verwendet (transparent) die bestmögliche Kombination an verfügbarer Hardware (z.B. InfiniBand + Shared Memory)

¹ <https://openucx.org>

² <https://openucx.readthedocs.io/en/master/api.html>



4.5 Zusammenfassung



Zusammenfassung

- Sockets sind eine Low-Level-Schnittstelle für die Kommunikation
 - Daher mühselig zu verwenden
- Richtig verwendet erlauben sie aber einen schnellen Nachrichtenaustausch
 - Die Anwendung weiß, wann welche Daten wo benötigt werden
- Zudem sind sie fast für alle Plattformen und Programmiersprachen verfügbar

